

Fresher Android

Kotlin OOP Advance





Generic |

Classes in Kotlin may have type parameters:

```
class Box<T>(t: T) {  
    var value = t  
}
```

In general, to create an instance of such a class, provide the type arguments:

```
val box: Box<Int> = Box<Int>(1  
val box = Box(1) // 1 has type Int, so the compiler figures out that it is Box<Int>)
```

Type parameters are placed *before* the name of the function:

```
fun <T> singletonList(item: T): List<T> {  
    // ...  
}
```

```
fun <T> T.basicToString(): String { // extension function  
    // ...  
}
```

To call a generic function, specify the type arguments at the call site *after* the name of the function:

```
val l = singletonList<Int>(1)
```

Type arguments can be omitted if they can be inferred from the context:

```
val l = singletonList(1)
```

- The most common type of constraint is an upper bound that corresponds to Java's extends keyword:
`fun <T : Comparable<T>> sort(list: List<T>) { ... }`
- The type specified after a colon is the *upper bound*: only a subtype of `Comparable<T>` can be substituted for `T`. For example:
`sort(listOf(1, 2, 3)) // OK. Int is a subtype of Comparable<Int>`
`sort(listOf(HashMap<Int, String>())) // Error`
- The default upper bound (if none specified) is `Any?`. Only one upper bound can be specified inside the angle brackets. If the same type parameter needs more than one upper bound, you need a separate `where` -clause:
`fun <T> copyWhenGreater(list: List<T>, threshold: T): List<String>`
 where `T : CharSequence,`
 `T : Comparable<T> {`
 return `list.filter { it > threshold }.map { it.toString() }`
 `}`
- The passed type must satisfy all conditions of the `where` clause simultaneously. In the above example, the `T` type must implement both `CharSequence` and `Comparable`.

Variance

TBD



Nested and Inner classes

2. Nested and Inner Classes

Classes can be nested in other classes:

```
class Outer {  
    private val bar: Int = 1  
    class Nested {  
        fun foo() = 2  
    }  
}
```

```
val demo = Outer.Nested().foo() // == 2
```

A nested class marked as inner can access the members of its outer class. Inner classes carry a reference to an object of an outer class:

```
class Outer {  
    private val bar: Int = 1  
    inner class Inner {  
        fun foo() = bar  
    }  
}
```

```
val demo = Outer().Inner().foo() // == 1
```




Enum classes

3. Enum Classes

The most basic usage of enum classes is implementing type-safe enums:

```
enum class Direction {  
    NORTH, SOUTH, WEST, EAST  
}
```

Since each enum is an instance of the enum class, it can be initialized as:

```
enum class Color(val rgb: Int) {  
    RED(0xFF0000),  
    GREEN(0x00FF00),  
    BLUE(0x0000FF)  
}
```

An enum class can implement an interface (but not derive from a class)

```
enum class IntArithmetics : BinaryOperator<Int>, IntBinaryOperator {  
    PLUS {  
        override fun apply(t: Int, u: Int): Int = t + u  
    },  
    TIMES {  
        override fun apply(t: Int, u: Int): Int = t * u  
    };  
  
    override fun applyAsInt(t: Int, u: Int) = apply(t, u)  
}
```




| Sealed classes



- *Sealed* classes and interfaces represent restricted class hierarchies that provide more control over inheritance.

```
sealed interface Expr
sealed class MathExpr(): Expr {
    abstract fun test()
}
data class Const(val number: Double) : MathExpr() {
    override fun test() {}
}
data class Sum(val e1: Expr, val e2: Expr) : MathExpr() {
    override fun test() {}
}
```

```
object NotANumber : Expr
```

- A sealed class is abstract by itself, it cannot be instantiated directly and can have abstract members

Sealed classes and when expression

- The key benefit of using sealed classes comes into play when you use them in a when expression

```
fun eval(expr: Expr): Double = when(expr) {  
    is Const -> expr.number  
    is Sum -> eval(expr.e1) + eval(expr.e2)  
    NotANumber -> Double.NaN  
    // the `else` clause is not required because we've covered all the cases  
}
```

- This works only if you use when as an expression (using the result) and not as a statement



Object expressions and declarations

5. Object Expressions and Declarations

Object expressions create objects of anonymous classes, that is, classes that aren't explicitly declared with the class declaration:

```
val helloWorld = object {  
    val hello = "Hello"  
    val world = "World"  
    // object expressions extend Any, so `override` is required on `toString()`  
    override fun toString() = "$hello $world"  
}
```

Note: Instances of anonymous classes are also called anonymous objects because they are defined by an expression, not a name.

Inheriting anonymous objects from supertypes:

```
window.addMouseListener(object : MouseAdapter() {  
    override fun mouseClicked(e: MouseEvent) { /*...*/ }  
  
    override fun mouseEntered(e: MouseEvent) { /*...*/ }  
})
```

```
object DataProviderManager {  
    fun registerDataProvider(provider: DataProvider) {  
        // ...  
    }  
  
    val allDataProviders: Collection<DataProvider>  
        get() = // ...  
}
```

- *object declaration* always has a name following the **object** keyword.
⇒ easy to declare singletons
- An *object declaration* inside a class can be marked with the **companion** keyword:

```
class MyClass {  
    companion object Factory {  
        fun create(): MyClass = MyClass()  
    }  
}
```

- There is one important semantic difference between object expressions and object declarations:
 - ❑ Object expressions are executed (and initialized) immediately, where they are used.
 - ❑ Object declarations are initialized lazily, when accessed for the first time.
 - ❑ A companion object is initialized when the corresponding class is loaded (resolved), matching the semantics of a Java static initializer



| Type alias



Type aliases

Type aliases provide alternative names for existing types:

```
typealias NodeSet = Set<Network.Node>
```

```
typealias FileTable<K> = MutableMap<K, MutableList<File>>
```

You can provide different aliases for function types:

```
typealias MyHandler = (Int, String, Any) -> Unit
```

```
typealias Predicate<T> = (T) -> Boolean
```

You can have new names for inner and nested classes:

```
class A {  
    inner class Inner  
}  
typealias Alnner = A.Inner
```

Lesson Summary

- Generics
- Nested and Inner Classes
- Enum Classes
- Sealed Classes
- Object Expressions and Declarations
- Type aliases

Thank you

